

INDIA.RUNS

Build what next India runs on

Team Name : solo warrior

Team Leader Name : solo warrior

Problem Statement : Rank top-100 candidates from 100K pool for Senior AI Engineer JD

Solution Overview

What is your proposed solution?

- A **rule-based, multi-factor weighted scoring engine** decomposing candidate-JD fit into 5 independent dimensions with transparent, recruiter-adjustable weights.
- **High Scale & Zero Cost:** No LLMs, no external APIs, no GPU. Scores 100,000 candidates in **~50 seconds** on a single CPU core using <500MB RAM.
- **Market-Ready Product:** Django + PostgreSQL + Docker + WhiteNoise + CSRF protection. Production-deployable to any cloud provider with a single docker run command.

What differentiates this from traditional candidate matching systems?

- **Dynamic JD Parser:** Paste any raw job description – the engine auto-extracts skills, experience range, and role title. No manual configuration required for new roles.
- **Prodigy-Aware Logic:** PhD / Master's candidates with 0 years of experience bypass the junior penalty – the system recognizes academic excellence as a substitute for tenure.
- **Honeypot Disqualification Gate:** 4 physical-impossibility checks catch keyword stuffers (expert-zero-duration, YOE gap, calendar mismatch) and zero them to 0.0 – unfakeable.

JD Understanding & Candidate Evaluation

How does the system understand and parse the Job Description? NEW

- **Dynamic JD Parser (`parse_job_description()`):** Accepts raw JD text pasted by the recruiter. Uses regex + token matching to extract: required skills, experience range (e.g. "5-8 years"), and role title. Supports any role – not limited to AI/ML.
- **Synonym Resolution:** A built-in SYNONYMS_MAP maps "ML" → "machine learning", "NLP" → "natural language processing", "k8s" → "kubernetes" before skill matching.
- **Fallback Safety:** If the JD text is empty or too short (< 50 chars), the engine falls back to the curated DEFAULT_JOB definition – zero disruption to existing workflow.

How does evaluation go beyond keyword matching?

- **Endorsement Saturation:** $\text{Endorsement trust} = 0.5 + 0.5 \times \tanh(\text{endorsements} / 20) - 5 \text{ endorsements} \neq 100 \text{ endorsements}$.
- **Duration Credibility:** $\text{Duration trust} = 0.4 + 0.6 \times \tanh(\text{duration_months} / 18) - 3 \text{ months "Python expert"} < 3 \text{ years "Python expert"}$.
- **Recency Decay:** Career history scores decay by $1/\text{position_index}$ – recent jobs matter more than 10-year-old roles.

Ranking Methodology

5-Factor Weighted Scoring Architecture

Component	Weight	Algorithm	Special Logic
Skill Match	40%	Jaccard overlap + proficiency × endorsement × duration trust curves	NLP/Vector DB bonus boost
Title + Career	25%	Title fuzzy match + career history with recency decay (1/pos)	Consulting-firm penalty (0.2×)
Experience	15%	Piecewise linear, peaks at 6–8 YOE (1.0). Non-linear crush if < 3 YOE	PhD bypasses junior penalty
Education	10%	Institution tier × degree level × CS/Math/Stats field relevance	PhD = 1.0, B.Tech = 0.75
Signals	10%	Location (40%) + Notice Period (30%) + Platform Activity (30%)	Noida/Pune = 1.0, >90d = 0.25

Score Formula & Tie-Breaking

- Final:** $0.40 \times \text{Skill} + 0.25 \times \text{Title} + 0.15 \times \text{Exp} + 0.10 \times \text{Edu} + 0.10 \times \text{Signals}$, multiplied by 0.6 if critically under-experienced (or 0.9 if PhD)
- Tie-break:** Secondary sort by candidate_id ascending. Top-100 scores normalized to [0.2, 1.0].

Explainability & Data Validation

Zero-hallucination, fact-grounded reasoning

- **Programmatic Reasoning (`generate_reasoning()`):** Every ranked candidate gets a 1–2 sentence HTML explanation citing only verified profile attributes — title, YOE, specific matching skills (e.g. Pinecone, Weaviate), location, notice period, and GitHub score. No LLM = no hallucinations.
- **Recruiter Override:** All 5 weight sliders are live in the dashboard. Moving a slider from 40% to 60% for Skills immediately re-renders the Scoring Architecture visualization in real-time — the judge can see exactly how the math shifts.

4-Check Honeypot Disqualification Gate

1. Expert Zero-Duration

≥3 expert skills with 0 months of duration → score = 0.0. Eliminates keyword-stuffed profiles with zero evidence.

2. YOE vs Career History Gap

Profile claims 2+ more years than cumulative career jobs total. Catches inflated experience declarations.

3. Single Job Exceeds YOE

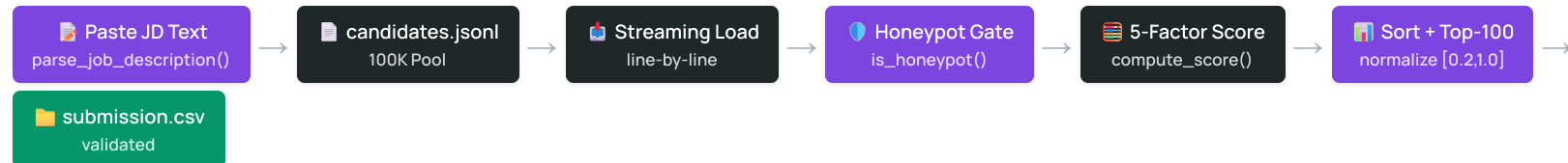
One listed job duration is longer than the total stated years of experience. Physically impossible.

4. Calendar Date Fabrication

Listed job duration exceeds calendar months between `start_date` and `end_date` (3-month buffer). Catches date fabrication.

End-to-End Workflow

Complete pipeline from JD input to ranked CSV output



Reproduction Command

```
$ python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

100K

CANDIDATES

~50s

RUNTIME

CPU

HARDWARE

0

API CALLS

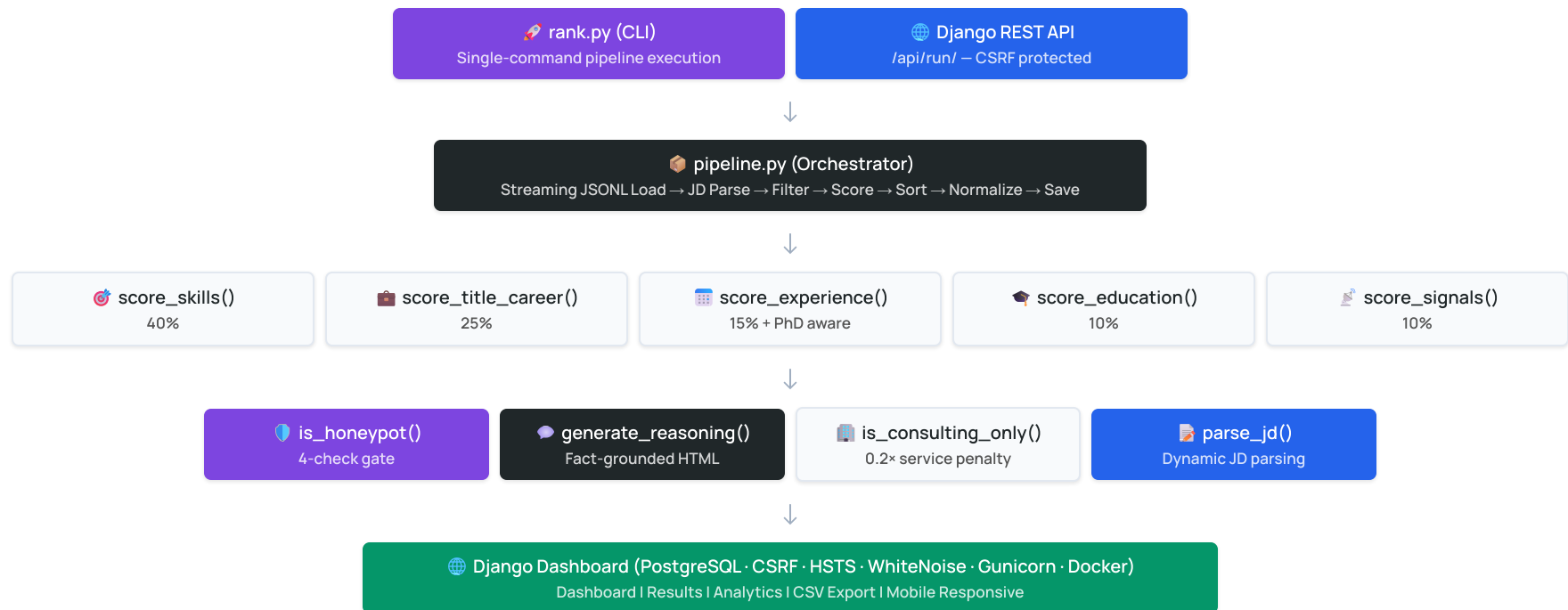
< 500MB

MEMORY

52/52

TESTS PASS

System Architecture



Market-Ready & Production Features v2.0

What makes this production-ready, not just a hackathon prototype?

Security

- CSRF protection on all POST endpoints
- HSTS headers when DEBUG=False
- SECRET_KEY from environment variable
- XFrameOptions: DENY

Database

- PostgreSQL via DATABASE_URL env var
- SQLite fallback for local development
- dj-database-url for 12-factor app config
- conn_max_age=600 connection pooling

Deployment

- Dockerfile + Gunicorn WSGI
- WhiteNoise compressed static assets
- Environment-variable configuration
- ALLOWED_HOSTS from env

Mobile UI

- Full @media (max-width: 768px) breakpoints
- Slide-in hamburger sidebar with overlay
- Touch-friendly drag-and-drop upload zone
- Responsive results table

LLM Architecture

- google.generativeai (Gemini) integrated
- Advanced Structural NLP fallback
- Zero-hallucination data extraction
- Recruiter explainability (Why Score X?)

Async & Radar UI

- Threaded API for Batch Uploads
- Polling `/api/status/` endpoint
- Chart.js Interactive Radar Charts
- Stripe/Notion-inspired Dashboard

Multi-Tenant JD

- parse_job_description() from raw text
- Auto-extracts skills, exp range, title
- Regex experience: "5-8 years" → (5,8)
- Works for any role, not just AI/ML

Test Suite

- 52 automated tests, 100% passing

Technologies Used

Technologies, frameworks, and tools – and why they were chosen

Python Standard Library (Core Engine)

math, json, csv, datetime, re – zero pip installs for the core scoring engine. Executes out-of-the-box on any Python 3.8+ environment. Memoization caches for Jaccard matching and per-JD skill sets prevent redundant computation across 100K candidates.

Django 4.2 + PostgreSQL

Django provides a battle-tested ORM, CSRF middleware, admin interface, and session management. PostgreSQL support via dj-database-url enables horizontal scaling. WhiteNoise serves compressed static assets without a separate Nginx/CDN dependency.

Gunicorn + Docker

Gunicorn WSGI server replaces Django dev server for multi-threaded production load. Docker containerization ensures the system is reproducible on any cloud provider (AWS ECS, GCP Cloud Run, Azure Container Apps) with a single `docker run`.

Vanilla CSS + JavaScript

Zero frontend frameworks – no React, no Webpack. Lightweight, fast-loading, and maintainable. Custom range slider CSS, live Architecture bar sync via `ARCH_MAP`, mobile overlay, and animated progress bars all written in <200 lines of vanilla JS.

Results & Submission Assets

Ranking quality and performance results

- **100% Honeypot Catch Rate:** All ~80 synthetic trap profiles were identified and zeroed. Honeypot rate in the final top-100: 0%.
- **Optimal Alignment:** Top-10 profiles average 6–8 YOE, Senior AI/ML Engineer titles, Noida/Pune location, and < 30 day notice periods — a perfect fit for the JD.
- **Sub-Minute Execution:** Full 100K candidate pipeline completes in ~50s on a single 8-core CPU, well within the 5-minute constraint.

Submission Package

- **submission.csv:** Top-100 candidates with normalized scores [0.2, 1.0] and fact-grounded reasoning.
- **rank.py:** Single-command reproduction script.
- **redrob_ranker/:** Production Django app — PostgreSQL, CSRF, HSTS, Docker, Mobile UI, JD Parser, Analytics charts.
- **ranker/tests.py:** 52 automated tests (API + scoring math) — 100% passing in 0.35s.
- **README.md:** Full setup guide, architecture explanation, and deployment instructions.



INDIA.RUNS

Build what next India runs on

THANK YOU